Figure 6.3 A more compact representation of a trie. This example trie contains the same elements as the binary search tree in figure 6.1.

Unlike k-ary search trees, though, no node in the tree actually stores the key associated with it. Instead, the characters held in the trie are actually stored in the edges between a node and its children.

Listing 6.1 illustrates a possible implementation of a `Trie` class (in an object-oriented pseudocode); you can also take a look at a full implementation on the book's repo on GitHub. In the simplest version, trie's nodes can only hold a tiny piece of information, just `true` or `false`: when marked with `true`, a node `N` is called a "*key node*", because it means that the sequence of characters in the edges of the path from the root to `N` corresponds to a word that is actually stored in the trie.

Nodes holding `false`, instead, are called "intermediate nodes", because they correspond to intermediate characters of one or more words stored in the trie.

Listing 6.1 class Trie

```
#type Char[]
Alphabet

class Node
  #type boolean
  keyNode

  #type HashMap<Char, Node>
  children
```